

本小节内容

二叉排序树删除实战

二叉排序树删除实战

代码实战步骤：在原有二叉树排序树建树，查找的基础上，新增了二叉排序树删除。二叉排序树的删除我们使用递归来实现，理解代码时可以结合

<https://www.cs.usfca.edu/~galles/visualization/BST.html> 动画网站。

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef int KeyType;
```

```
typedef struct BSTNode{
```

```
    KeyType key;
```

```
    struct BSTNode *lchild,*rchild;
```

```
}BSTNode,*BiTree;
```

```
//54,20,66,40,28,79,58
```

```
// 王道书上的递归写法，代码简单，但是理解有难度
```

```
//int BST_Insert(BiTree &T,KeyType k)
```

```
//{
```

```
//    if(NULL==T)
```

```
//    {    //为新节点申请空间，第一个结点作为树根，后面递归再进入的不是树根，是为叶子结点
```

```
//        T=(BiTree)malloc(sizeof(BSTNode));
```

```
//        T->key=k;
```

```
//        T->lchild=T->rchild=NULL;
```

```
//        return 1;//代表插入成功
```

```
//    }
```

```
//    else if(k==T->key)
```

```
//        return 0;//发现相同元素，就不插入
```

```
//    else if(k<T->key)//如果要插入的结点，小于当前结点
```

```
//        //函数调用结束后，左孩子和原来的父亲会关联起来，巧妙利用了引用机制
```

```
//        return BST_Insert(T->lchild,k);
```

```
//    else
```

```
//        return BST_Insert(T->rchild,k);
```

```
//}
```

```
//54,20,66,40,28,79,58
```

```
int BST_Insert(BiTree& T, KeyType k)
```

```
{
```

```
if (NULL == T)
{
    //为新节点申请空间，第一个结点作为树根，T 是树根
    T = (BiTree)malloc(sizeof(BSTNode));
    T->key = k;
    T->lchild = T->rchild = NULL;
    return 1; //代表插入成功
}
BiTree p = T, parent; //p 用来遍历, parent 用来存 p 的父亲
while (p)
{
    parent = p;
    if (k == p->key)
    {
        return 0;
    }
    else if (k < p->key)
    {
        p = p->lchild;
    }
    else {
        p = p->rchild;
    }
}
//开始申请空间并插入
BiTree pnew = (BiTree)malloc(sizeof(BSTNode));
pnew->key = k;
pnew->lchild = pnew->rchild = NULL;
if (k > parent->key)
{
    parent->rchild = pnew; //新结点在父亲的右边
}
else {
    parent->lchild = pnew; //新结点在父亲的左边
}
}
//创建二叉排序树
void Creat_BST(BiTree &T, KeyType str[], int n)
{
    T = NULL; //T 是树根
    int i = 0;
    while (i < n)
    {
        BST_Insert(T, str[i]); //把某一个结点放入二叉查找树
    }
}
```

```
        i++;
    }
}
```

//也可以用递归实现，只不过循环查找已经非常简单清晰，用递归必要性不强

```
BSTNode *BST_Search(BiTree T,KeyType key,BiTree &p)
{
    p=NULL;//存储要找的结点的父亲
    while(T!=NULL&&key!=T->key)
    {
        p=T;
        if(key<T->key) T=T->lchild;//比当前节点小，就左边找
        else T=T->rchild;//比当前节点大，右边去
    }
    return T;
}
```

//这个书上没有二叉排序树删除代码——考大题没那么高

```
void DeleteNode(BiTree &root,KeyType x){
    if(root == NULL){
        return;
    }
    if(root->key>x){
        DeleteNode(root->lchild,x);//往左子树找要删除的结点
    }else if(root->key<x){
        DeleteNode(root->rchild,x);//往右子树找要删除的结点
    }else{ //查找到了删除结点
        if(root->lchild == NULL){ //左子树为空，右子树直接顶上去
            BiTree tempNode = root;//用临时的存着的目的是会要 free
            root = root->rchild;
            free(tempNode);
        }else if(root->rchild == NULL){ //右子树为空，左子树直接顶上去
            BiTree tempNode = root;//临时指针
            root = root->lchild;
            free(tempNode);
        }else{ //左右子树都不为空
            //一般的删除策略是左子树的最大数据 或 右子树的最小数据 代替要删除的结点(这里
            采用查找左子树最大数据来代替)
            BiTree tempNode = root->lchild;
            while(tempNode->rchild!=NULL){ //向右找到最大的
                tempNode = tempNode->rchild;
            }
            root->key = tempNode->key;//把 tempNode 对应的值替换到要删除的值的位上
            DeleteNode(root->lchild,tempNode->key);//删除 tempNode
        }
    }
}
```

```
}
```

```
void InOrder(BiTree T)
```

```
{
```

```
    if(T!=NULL)
```

```
    {
```

```
        InOrder(T->lchild);
```

```
        printf("%3d",T->key);
```

```
        InOrder(T->rchild);
```

```
    }
```

```
}
```

```
//《王道 C 督学营》课程
```

```
//二叉排序树的创建，中序遍历，查找，删除
```

```
int main()
```

```
{
```

```
    BiTree T=NULL; //树根
```

```
    BiTree parent; //存储父亲结点的地址值
```

```
    BiTree search;
```

```
    KeyType str[7]={54,20,66,40,28,79,58}; //将要进入二叉排序树的元素值
```

```
    Creat_BST(T,str,7);
```

```
    InOrder(T);
```

```
    printf("\n");
```

```
    search=BST_Search(T,40,parent);
```

```
    if(search)
```

```
    {
```

```
        printf("找到对应结点， 值=%d\n",search->key);
```

```
    }else{
```

```
        printf("未找到对应结点\n"); //没找到 search 返回的是 NULL
```

```
    }
```

```
    DeleteNode(T,40); //删除某个结点
```

```
    InOrder(T);
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```